

Proyecto Final | 3er entrega

alumno : José Soria Díaz
comisión : 53185

temática : Fotografía
entrega : Mayo 2024

1 Introducción

- Un proyecto comercial web llamado Fotografía se pone en marcha, el proyecto cuenta con varias áreas de desarrollo pero hoy nos enfocaremos en la gestión de base de datos y un breve análisis de la evolución del proyecto en alguna temática en particular, desde la fundación del proyecto en inicios del 2023.

2 Objetivo

- Se desea diseñar un esquema de base de datos a fin de poder gestionar una plataforma web de fotografía. Dicho esquema podrá almacenar información estática por ejemplo datos de contacto, e información dinámica, interacciones con comentarios, órdenes de compra, pagos, etc...
- La base de datos debe permitir la generación y creación de reportería para el análisis de la evolución del proyecto comercial.

3 Problemática

- Poder gestionar un sitio con usuarios de todo el mundo. Aunque inicialmente el modelo de negocios inició con mercado en pocos países de Latam, hoy ya se encuentra disponible en Argentina, Uruguay, Chile, Brasil, Colombia, México y Estados Unidos. En el futuro será abierto a otros diferentes mercados.

4 Modelo de negocio

- Un portal comercial en donde solo los fotógrafos puedan exponer sus trabajos personales y al mismo tiempo pueda unir a fotógrafos y usuarios como compradores de su contenido.
- Los fotógrafos disponen sus fotografías en álbumes. El usuario podrá comprar el contenido del álbum y este estará disponible para su descarga en alta resolución. Al mismo tiempo el usuario podrá ponerse en contacto para consultar y requerir servicio profesional de su fotógrafo preferido para el estilo o proyecto que necesite. El fotógrafo publicará sus álbumes/imágenes que considere según su estilo de preferencia. Los elementos que son puestos a la venta son los álbumes, pudiéndose comprar el álbum con todo su contenido y no así fotografías individuales. La interactividad sí puede establecerse a nivel de fotografía.

DIAGRAMA ENTIDAD RELACIÓN - DER

5 Diagrama de Entidad-Relación

- Se define un esquema básico para implementar en la gestión de la plataforma web, en la que se divide en 4 capas generales:

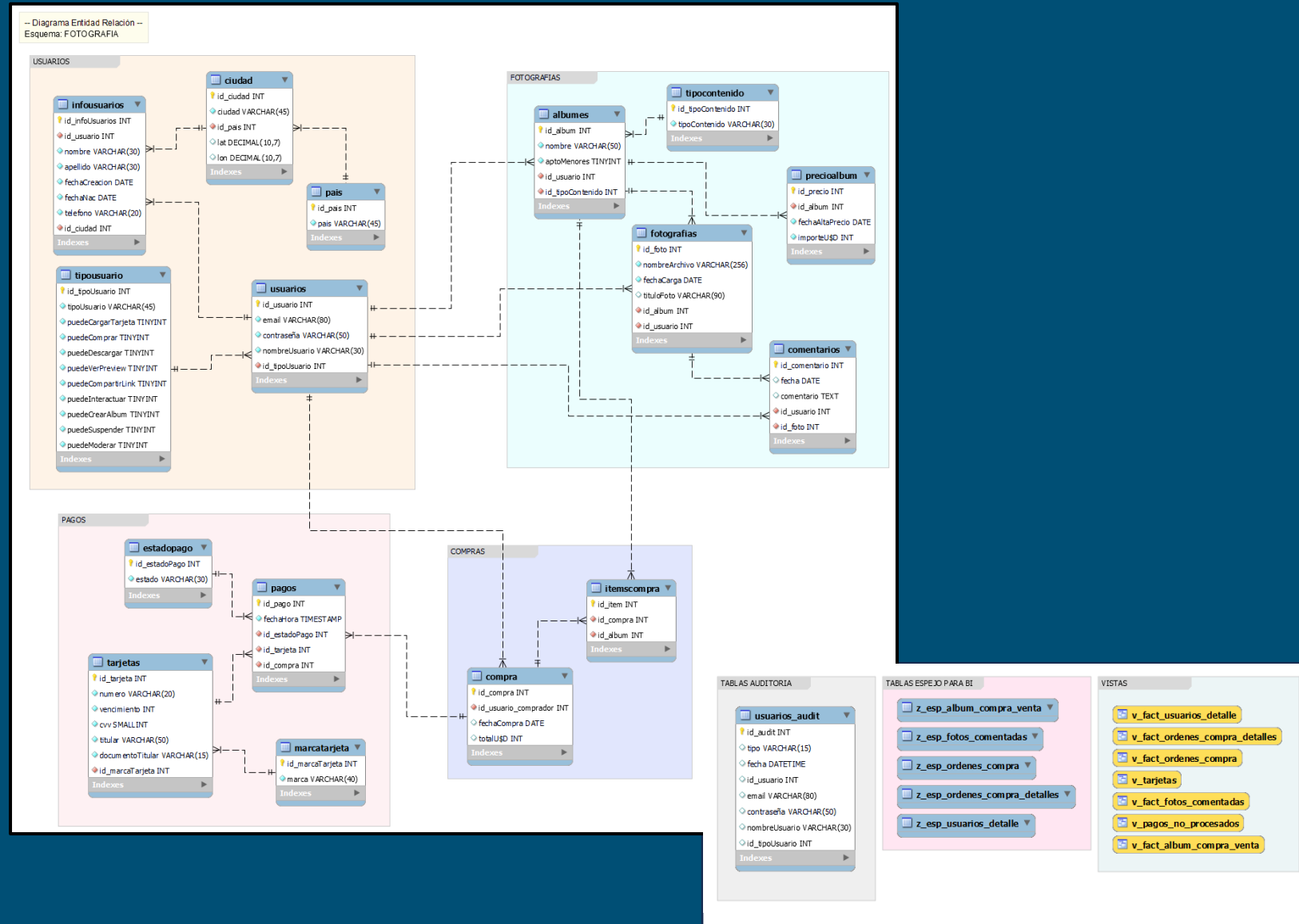
Usuarios,
Fotografías,
Compras,
Pagos

- Se cuenta con los siguientes objetos:

16 tablas
1 tabla de auditoría
5 tablas espejo para BI
7 vistas
6 funciones
5 stored procedures
3 triggers

Utilice el link de descarga para acceder al contenido del proyecto.

[{descargar}](#)



TABLAS

6 Listado de Tablas (a)

Capa USUARIOS

Tabla	Descripción: Dimensión Listado de países							
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
pais	id_pais	int		NO	PRIMARY	INDEX	auto_increment	
	pais	varchar	45	NO	UNIQUE	INDEX		País del usuario registrado. Unico evita repeticiones

Tabla	Descripción: Dimensión Listado de ciudades							
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
ciudad	id_ciudad	int		NO	PRIMARY	INDEX	auto_increment	
	ciudad	varchar	45	NO				Ciudad del usuario registrado
	id_pais	int		NO	FOREIGN	INDEX		
	lat	DECIMAL	(10,7)					coordenadas geograficas de latitud
	lon	DECIMAL	(10,7)					coordenadas geograficas de longitud

Tabla	Descripción: Tabla con tipos de usuarios y sus permisos para interactuar en la plataforma							
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
tipoUsuario	id_tipoUsuario	int		NO	PRIMARY	INDEX	auto_increment	
	tipoUsuario	varchar	45	NO				admin; usuario; visitante; fotografo; etc
	puedeCargarTarjeta	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeComprar	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeDescargar	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeVerPreview	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeCompartirLink	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeInteractuar	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeCrearAlbum	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeSuspender	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma
	puedeModerar	tinyint		NO				columnas con valores 1 o 0, indican acciones habilitadas en la plataforma

```
CREATE TABLE pais (
  id_pais int NOT NULL AUTO_INCREMENT,
  pais varchar(45) NOT NULL,
  PRIMARY KEY (id_pais),
  UNIQUE KEY pais (pais)
);
```

```
CREATE TABLE ciudad (
  id_ciudad int NOT NULL AUTO_INCREMENT,
  ciudad varchar(45) NOT NULL,
  id_pais int NOT NULL,
  lat decimal(10,7) DEFAULT NULL,
  lon decimal(10,7) DEFAULT NULL,
  PRIMARY KEY (id_ciudad),
  KEY ciudad_ix (id_pais),
  CONSTRAINT ciudad_fk FOREIGN KEY (id_pais) REFERENCES pais (id_pais)
);
```

```
CREATE TABLE tipousuario (
  id_tipoUsuario int NOT NULL AUTO_INCREMENT,
  tipoUsuario varchar(45) NOT NULL,
  puedeCargarTarjeta tinyint NOT NULL,
  puedeComprar tinyint NOT NULL,
  puedeDescargar tinyint NOT NULL,
  puedeVerPreview tinyint NOT NULL,
  puedeCompartirLink tinyint NOT NULL,
  puedeInteractuar tinyint NOT NULL,
  puedeCrearAlbum tinyint NOT NULL,
  puedeSuspender tinyint NOT NULL,
  puedeModerar tinyint NOT NULL,
  PRIMARY KEY (id_tipoUsuario)
);
```

6 Listado de Tablas (b)

Tabla	Descripción: Información del perfil del usuario registrado							
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
infoUsuarios	id_info_usuario	int		NO	PRIMARY	INDEX	auto_increment	
	id_usuario	int		NO	FOREIGN	INDEX		
	nombre	varchar	30	NO				
	apellido	varchar	30	NO				
	fechaCreacion	date		NO				fecha de registraci3n del usuario
	fechaNac	date		NO				fecha de nacimiento del usuario registrado
	telefono	varchar	20	NO				tel3fono de contacto
	id_ciudad	int		NO	FOREIGN	INDEX		

Tabla	Descripción: Tabla con datos del usuario registrado, credenciales de logueo y tipo de usuario							
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
usuarios	id_usuario	int		NO	PRIMARY	INDEX	auto_increment	
	email	varchar	80	NO	UNIQUE	INDEX		email de registracion, sirve en el logueo a la plataforma
	contraseña	varchar	50	NO				
	nombreUsuario	varchar	30	NO	UNIQUE	INDEX		nombre de usuario, sirve para realizar comentarios y otras interacciones en el sitio
	id_tipoUsuario	int		NO	FOREIGN	INDEX		

```
CREATE TABLE infousuarios (
  id_infoUsuarios int NOT NULL AUTO_INCREMENT,
  id_usuario int NOT NULL,
  nombre varchar(30) NOT NULL,
  apellido varchar(30) NOT NULL,
  fechaCreacion date NOT NULL,
  fechaNac date NOT NULL,
  telefono varchar(20) NOT NULL,
  id_ciudad int NOT NULL,
  PRIMARY KEY (id_infoUsuarios),
  KEY id_usuario_ix (id_usuario),
  KEY id_ciudad_ix (id_ciudad),
  CONSTRAINT infousuarios_fk_1 FOREIGN KEY (id_usuario) REFERENCES usuarios (id_usuario),
  CONSTRAINT infousuarios_fk_2 FOREIGN KEY (id_ciudad) REFERENCES ciudad (id_ciudad)
);
```

```
CREATE TABLE usuarios (
  id_usuario int NOT NULL AUTO_INCREMENT,
  email varchar(80) NOT NULL,
  contraseña varchar(50) NOT NULL,
  nombreUsuario varchar(30) NOT NULL,
  id_tipoUsuario int NOT NULL,
  PRIMARY KEY (id_usuario),
  UNIQUE KEY nombreUsuario (nombreUsuario),
  UNIQUE KEY email (email),
  KEY id_tipoUsuario_ix (id_tipoUsuario),
  CONSTRAINT usuarios_fk_1 FOREIGN KEY (id_tipoUsuario) REFERENCES tipousuario (id_tipoUsuario)
);
```

6 Listado de Tablas (c)

Capa FOTOGRAFIAS

Tabla	Descripción:		Dimensión Tabla con tipo de contenido publicado					
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
tipoContenido	id_tipoContenido	int		NO	PRIMARY	INDEX		
	tipoContenido	varchar	30	NO				Estilo fotográfico, retratos; paisajes; budoir; producto; periodismo; reportajes; inmobiliario; etc

Tabla	Descripción:		Listado de comentarios realizados en las fotografías					
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
comentarios	id_comentario	int		NO	PRIMARY	INDEX		
	fecha	date		YES				fecha del comentario
	comentario	text		YES				
	id_usuario	int		NO	FOREIGN	INDEX		
	id_foto	int		NO	FOREIGN	INDEX		

Tabla	Descripción:		Listado de los importes a la venta de cada álbum fotográfico					
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
precioAlbum	id_precio	int		NO	PRIMARY	INDEX		
	id_album	int		NO	FOREIGN	INDEX		
	fechaAltaPrecio	date		NO				cada álbum puede tener precios que cambien con el tiempo, para la venta se toma el último cargado y a su vez se puede modificar para proximas ventas sin interferir en los registros ya vendidos
	importeUSD	int		NO				misma consideración que fechaAltaPrecio

```
CREATE TABLE tipocontenido (
  id_tipoContenido int NOT NULL,
  tipoContenido varchar(30) NOT NULL,
  PRIMARY KEY (id_tipoContenido)
);
```

```
CREATE TABLE comentarios (
  id_comentario int NOT NULL,
  fecha date DEFAULT NULL,
  comentario text,
  id_usuario int NOT NULL,
  id_foto int NOT NULL,
  PRIMARY KEY (id_comentario),
  KEY id_usuario_ix (id_usuario),
  KEY id_foto_ix (id_foto),
  CONSTRAINT comentarios_fk_1 FOREIGN KEY (id_usuario) REFERENCES usuarios (id_usuario),
  CONSTRAINT comentarios_fk_2 FOREIGN KEY (id_foto) REFERENCES fotografias (id_foto)
);
```

```
CREATE TABLE precioalbum (
  id_precio int NOT NULL,
  id_album int NOT NULL,
  fechaAltaPrecio date NOT NULL,
  importeUSD int NOT NULL,
  PRIMARY KEY (id_precio),
  KEY id_album_ix (id_album),
  CONSTRAINT precioalbum_fk_1 FOREIGN KEY (id_album) REFERENCES albums (id_album)
);
```


6 Listado de Tablas (d)

Tabla	Descripción:		Detalle de las fotografías cargadas por fotógrafos					
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
fotografias	id_foto	int		NO	PRIMARY	INDEX		
	nombreArchivo	varchar	256	NO				nombre de archivo físico alojado en servidor
	fechaCarga	date		NO				
	tituloFoto	varchar	90	YES				Optativo, título de la imagen proporcionado por fotógrafo
	id_album	int		NO	FOREIGN	INDEX		
	id_usuario	int		NO	FOREIGN	INDEX		

Tabla	Descripción:		Detalle de álbumes contenedores, agrupaciones de fotografías subidas por profesionales					
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
albumes	id_album	int		NO	PRIMARY	INDEX		
	nombre	varchar	50	NO				nombre de álbum o contenedor de fotografías
	aptoMenores	tinyint		NO				flag o marca publicada en plataforma, si el tipo de usuario no tiene habilitado el flag no podrá ver el contenido del álbum (ver tabla tipoUsuario, campo puedeVerPreview)
	id_usuario	int		NO	FOREIGN	INDEX		
	id_tipoContenido	int		NO	FOREIGN	INDEX		

```
CREATE TABLE fotografias (
  id_foto int NOT NULL,
  nombreArchivo varchar(256) NOT NULL,
  fechaCarga date NOT NULL,
  tituloFoto varchar(90) DEFAULT NULL,
  id_album int NOT NULL,
  id_usuario int NOT NULL,
  PRIMARY KEY (id_foto),
  KEY id_album_ix (id_album),
  KEY id_usuario_ix (id_usuario),
  CONSTRAINT fotografias_fk_1 FOREIGN KEY (id_album) REFERENCES albumes (id_album),
  CONSTRAINT fotografias_fk_2 FOREIGN KEY (id_usuario) REFERENCES usuarios (id_usuario)
);
```

```
CREATE TABLE albumes (
  id_album int NOT NULL,
  nombre varchar(50) NOT NULL,
  aptoMenores tinyint NOT NULL,
  id_usuario int NOT NULL,
  id_tipoContenido int NOT NULL,
  PRIMARY KEY (id_album),
  KEY id_usuario_ix (id_usuario),
  KEY id_tipoContenido_ix (id_tipoContenido),
  CONSTRAINT albumes_fk_1 FOREIGN KEY (id_usuario) REFERENCES usuarios (id_usuario),
  CONSTRAINT albumes_fk_2 FOREIGN KEY (id_tipoContenido) REFERENCES tipocontenido (id_tipoContenido)
);
```

6 Listado de Tablas (e)

Capa COMPRAS

Tabla		Descripción: Detalle de items agregados al carrito de compras						
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
itemsCompra	id_item	int		NO	PRIMARY	INDEX		
	id_compra	int		NO	FOREIGN	INDEX		
	id_album	int		NO	FOREIGN	INDEX		

Tabla		Descripción: Tabla de compra/facturación final						
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
compra	id_compra	int		NO	PRIMARY	INDEX		
	id_usuarioComprador	int		NO	FOREIGN	INDEX		
	fechaCompra	date		YES				
	totalUSD	int		YES				

```
CREATE TABLE itemscompra (  
  id_item int NOT NULL,  
  id_compra int NOT NULL,  
  id_album int NOT NULL,  
  PRIMARY KEY (id_item),  
  KEY id_album_ix (id_album),  
  KEY id_compra_ix (id_compra),  
  CONSTRAINT itemscompra_fk_1 FOREIGN KEY (id_compra) REFERENCES compra (id_compra),  
  CONSTRAINT itemscompra_fk_2 FOREIGN KEY (id_album) REFERENCES albums (id_album)  
);
```

```
CREATE TABLE compra (  
  id_compra int NOT NULL,  
  id_usuario_comprador int NOT NULL,  
  fechaCompra date DEFAULT NULL,  
  totalUSD int DEFAULT NULL,  
  PRIMARY KEY (id_compra),  
  KEY id_usuario_ix (id_usuario_comprador),  
  CONSTRAINT compra_fk_1 FOREIGN KEY (id_usuario_comprador) REFERENCES usuarios (id_usuario)  
);
```

6 Listado de Tablas (f)

Capa PAGOS

Tabla Descripción: Dimensión Tabla de los posibles estados de un pago								
estadoPago	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
	id_estadoPago	int		NO	PRIMARY			
	estado	varchar	30	NO				estado del pago: aprobado; pendiente; rechazado; error

Tabla Descripción: Dimensión Tabla de marcas de tarjetas de crédito								
marcaTarjeta	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
	id_marcaTarjeta	int		NO	PRIMARY	INDEX	auto_increment	
	marca	varchar	40	NO	UNIQUE	INDEX		marca de tarjeta: visa; mastercard; amex; etc

Tabla Descripción: Detalle de información de tarjetas de crédito usadas para abonar								
tarjetas	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
	id_tarjeta	int		NO	PRIMARY	INDEX	auto_increment	
	numero	varchar	16	NO				
	vencimiento	int		NO				
	cvv	smallint		NO				codigo de seguridad 3 digitos
	titular	varchar	50	NO				titular de la tarjeta de crédito
	documentoTitular	varchar	15	NO				
	id_marcaTarjeta	int		NO	FOREIGN	INDEX		

Tabla Descripción: Detalle del pago								
pagos	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
	id_pago	int		NO	PRIMARY	INDEX		
	fechaHora	timestamp		NO				
	id_estadoPago	int		NO	FOREIGN	INDEX		
	id_tarjeta	int		NO	FOREIGN	INDEX		
	id_compra	int		NO	FOREIGN	INDEX		

```
CREATE TABLE estadopago (  
  id_estadoPago int NOT NULL,  
  estado varchar(30) NOT NULL,  
  PRIMARY KEY (id_estadoPago)  
);
```

```
CREATE TABLE marcatarjeta (  
  id_marcaTarjeta int NOT NULL AUTO_INCREMENT,  
  marca varchar(40) NOT NULL,  
  PRIMARY KEY (id_marcaTarjeta),  
  UNIQUE KEY marca (marca)  
);
```

```
CREATE TABLE tarjetas (  
  id_tarjeta int NOT NULL AUTO_INCREMENT,  
  numero varchar(20) NOT NULL,  
  vencimiento int NOT NULL,  
  cvv smallint NOT NULL,  
  titular varchar(50) NOT NULL,  
  documentoTitular varchar(15) NOT NULL,  
  id_marcaTarjeta int NOT NULL,  
  PRIMARY KEY (id_tarjeta),  
  KEY id_marcaTarjeta_ix (id_marcaTarjeta),  
  CONSTRAINT tarjetas_fk_1 FOREIGN KEY (id_marcaTarjeta)  
    REFERENCES marcatarjeta (id_marcaTarjeta)  
);
```

```
CREATE TABLE pagos (  
  id_pago int NOT NULL,  
  fechaHora timestamp NOT NULL,  
  id_estadoPago int NOT NULL,  
  id_tarjeta int NOT NULL,  
  id_compra int NOT NULL,  
  PRIMARY KEY (id_pago),  
  KEY id_estadoPago_ix (id_estadoPago),  
  KEY id_tarjeta_ix (id_tarjeta),  
  KEY id_compra_ix (id_compra),  
  CONSTRAINT pagos_fk_1 FOREIGN KEY (id_estadoPago) REFERENCES estadopago (id_estadoPago),  
  CONSTRAINT pagos_fk_2 FOREIGN KEY (id_tarjeta) REFERENCES tarjetas (id_tarjeta),  
  CONSTRAINT pagos_fk_3 FOREIGN KEY (id_compra) REFERENCES compra (id_compra)  
);
```

6 Listado de Tablas (g)

Capa Tabla de Auditoría

Tabla de auditoría sobre la tabla 'usuarios', se utilizará más adelante en la definición de los Triggers para un Insert, Update, y Delete de los registros de la tabla 'usuarios'.

Tabla		Descripción: Auditoría en tabla 'usuarios'						
	Columna	Tipo	Largo	NULL	KEY	INDEX	Otros	Anotaciones
usuarios_audit	id_audit	int		NO	PRIMARY	INDEX		
	tipo	varchar	15					
	fecha	datetime						
	id_usuario	int		NO				
	email	varchar	80	NO				
	contraseña	varchar	50	NO				
	nombreUsuario	varchar	30	NO				
	id_tipoUsuario	int		NO				

```
CREATE TABLE usuarios_audit (  
  id_audit INT NOT NULL AUTO_INCREMENT,  
  tipo VARCHAR(15),  
  fecha DATETIME,  
  id_usuario int NOT NULL,  
  email varchar(80) NOT NULL,  
  contraseña varchar(50) NOT NULL,  
  nombreUsuario varchar(30) NOT NULL,  
  id_tipoUsuario int NOT NULL,  
  PRIMARY KEY(id_audit)  
);
```

VISTAS

Proyecto Final | Vistas

7 Vistas (a)

El propósito en general de todas las vistas es poder aplicarlas como tablas de hecho para ser usadas en herramienta de visualización.

Info detallada de usuarios creados, y sus detalles personales asociados (no se muestra campo "contraseña").

VISTA			
v_fact_usuarios_detalle	tablas involucradas	campos	tipo/largo
	infousuarios	id_usuario	int
	usuarios	tipoUsuario	varchar(45)
	tipousuario	nombreUsuario	varchar(30)
	ciudad	email	varchar(80)
	país	nombre	varchar(30)
		apellido	varchar(30)
		fechaCreacion	date
		fechaNac	date
		telefono	varchar(20)
		ciudad	varchar(45)
		país	varchar(45)
		id_infoUsuarios	int
		id_tipoUsuario	int
		id_ciudad	int
		id_país	int

id_usuario	tipoUsuario	nombreUsuario	email	nombre	apellido	fechaCreacion	fechaNac	telefono	ciudad	país	id_infoUsuarios	id_tipoUsuario	id_ciudad	id_país
1	admin	equinox	jose.soria@photo.com	Jose	Soria	2023-01-01	1981-01-06	+54 (223) 170-2332	Mar del Plata	Argentina	1	1	124	9
2	admin	weigandtm	mabel.weigandt@photo.com	Mabel	Weigandt	2023-03-13	1980-09-09	+54 (223) 377-7093	Mar del Plata	Argentina	77	1	124	9
4	usuario	clewelyn1	ataillant1@google.co.jp	Jillie	Dickson	2023-09-05	1984-08-18	+86 (129) 774-6532	Joaquín V. González	Argentina	264	2	126	9
6	usuario	npietrowicz3	fboynton3@yellowpages.com	Melitta	Espadate	2023-07-03	1988-11-30	+254 (560) 643-5528	Luján de Cuyo	Argentina	188	2	86	9
7	usuario	sbertelsen4	jblacker4@blinklist.com	Branden	Truett	2023-03-06	1981-07-08	+234 (475) 909-5314	Obando	Colombia	69	2	683	42
8	usuario	plingsland5	kwattam5@chiron.com	Adi	Watters	2023-11-25	1981-10-12	+968 (642) 525-3735	Retiro	Argentina	330	2	81	9
9	usuario	cdorrling6	hdenyukhin6@deviantart.com	Christabel	Habbijam	2024-02-16	2004-12-05	+86 (662) 726-4806	Municipio de Copacabana	Colombia	425	2	709	42
10	usuario	tbarkworth7	usalerg7@freewebs.com	Hana	Elice	2023-08-10	1977-12-19	+48 (738) 309-5594	Buritzeiro	Brasil	237	2	336	26
15	usuario	jperviewc	mbackhurstc@ted.com	Vivienne	Scattergood	2023-09-26	1979-05-01	+86 (957) 341-8568	Elmira	Estados Unidos	280	2	814	60
16	usuario	kfoardd	lpickd@archive.org	Gisela	Loveman	2023-06-07	1976-10-24	+503 (182) 504-2356	Russas	Brasil	165	2	308	26

Vista que detalla los ítems o elementos que conforman las órdenes de compra con su precio por unidad (precio por álbum), usuario creador del álbum, tipo de contenido, fecha creación.

Funciones que participan: fPrecioAlbum()

VISTA			
v_fact_ordenes_compra_detalle	tablas involucradas	campos	tipo/largo
	itemscompra	id_compra	int
	albumes	fechaCompra	date
	usuarios	nombreAlbum	varchar(50)
	infousuarios	importeUSD	int
	tipocontenido	aptoMenores	tinyint
	ciudad	tipoContenido	varchar(30)
		país	fechaCreacionAlbum
	compra	fechaCreacionAlbum	date
		nombreFotografo	varchar(61)
		paísFotografo	varchar(45)
		nombreUsuarioFotografo	varchar(30)
		id_item	int
		id_album	int
		id_usuario_fotografo	int
		id_infoUsuario	int
		id_tipoContenido	int
		id_país_fotografo	int

id_compra	fechaCompra	nombreAlbum	importeUSD	aptoMenores	tipoContenido	fechaCreacionAlbum	nombreFotografo	paísFotografo	nombreUsuarioFotografo	id_item	id_album	id_usuario_fotografo	id_infoUsuario	id_tipoContenido
1	2023-02-15	Encuentros del Camino	28	1	producto	2024-03-20	Willard Chrystal	Colombia	tedgeona	39	110	13	430	4
1	2023-02-15	Espacios de Descanso	19	1	macro	2024-03-13	Marjory Kimmings	Argentina	lshirrellax	104	172	396	453	13
1	2023-02-15	Encuentros con la Luz	19	1	minimalismo	2023-07-31	Glyn Luckman	Estados Unidos	mharesnapebp	80	187	424	146	10
1	2023-02-15	Espacios de Serenidad	23	1	360°	2023-05-04	Amos Emlien	Estados Unidos	ltuteaz	201	227	398	96	16
2	2023-07-17	Belleza en Detalles	18	1	retratos	2024-01-19	Clemmie Duffit	Brasil	sdllintone71	97	58	256	384	1
2	2023-07-17	Horizontes en Armonía	24	1	retratos	2023-03-13	Minda Geeves	Argentina	pmusselwhiteh	127	90	20	30	1
2	2023-07-17	Horizontes en Armonía	30	1	producto	2024-04-27	Marjory Kimmings	Argentina	lshirrellax	214	134	396	453	4
2	2023-07-17	Latidos del Mundo	9	1	street	2023-11-09	Vi Reynolds	Chile	ewiddopaq	165	166	389	239	17
2	2023-07-17	Espacios de Libertad	23	1	street	2023-12-12	Chrisse Casine	Estados Unidos	mscholte9c	170	194	339	343	17
3	2023-07-28	Susurros del Viento	22	1	astro	2023-07-02	Wendye Davidovsky	Estados Unidos	krennebeck9h	209	35	344	147	11

```
CREATE OR REPLACE VIEW v_fact_usuarios_detalle AS
SELECT
```

```
    u.id_usuario,
    tu.tipoUsuario,
    u.nombreUsuario,
    u.email,
    iu.nombre,
    iu.apellido,
    iu.fechaCreacion,
    iu.fechaNac,
    iu.telefono,
    c.ciudad,
    p.país,
    iu.id_infoUsuarios,
    tu.id_tipoUsuario,
    c.id_ciudad,
    p.id_país
FROM infousuarios AS iu
INNER JOIN usuarios AS u ON iu.id_usuario = u.id_usuario
INNER JOIN tipousuario AS tu ON u.id_tipoUsuario = tu.id_tipoUsuario
INNER JOIN ciudad AS c ON iu.id_ciudad = c.id_ciudad
INNER JOIN país AS p ON p.id_país = c.id_país
;
```

```
CREATE OR REPLACE VIEW v_fact_ordenes_compra_detalle AS
SELECT
```

```
    ic.id_compra,
    c.fechaCompra,
    a.nombre AS nombreAlbum,
    fPrecioAlbum(ic.id_album) AS importeUSD,
    a.aptoMenores,
    tc.tipoContenido,
    (SELECT fo.fechaCarga
     FROM fotografias AS fo
     WHERE fo.id_album = ic.id_album
     GROUP BY fo.fechaCarga, fo.id_album
     LIMIT 1) AS fechaCreacionAlbum,
    concat(iu.nombre, " ", iu.apellido) AS nombreFotografo,
    p.país AS paísFotografo,
    u.nombreUsuario AS nombreUsuarioFotografo,
    ic.id_item,
    ic.id_album,
    a.id_usuario AS id_usuario_fotografo,
    iu.id_infoUsuarios AS id_infoUsuario,
    a.id_tipoContenido,
    p.id_país AS id_país_fotografo
FROM itemscompra AS ic
INNER JOIN albumes AS a ON ic.id_album = a.id_album
INNER JOIN usuarios AS u ON a.id_usuario = u.id_usuario
INNER JOIN infousuarios AS iu ON iu.id_usuario = u.id_usuario
INNER JOIN tipocontenido AS tc ON tc.id_tipoContenido = a.id_tipoContenido
INNER JOIN ciudad AS ciu ON ciu.id_ciudad = iu.id_ciudad
INNER JOIN país AS p ON p.id_país = ciu.id_país
INNER JOIN compra AS c ON c.id_compra = ic.id_compra
ORDER BY id_compra asc, a.id_album asc
;
```

7 Vistas (b)

Vista que muestra detalle de las órdenes de compra, con su importe total, detalles personales de usuario comprador.

VISTA	tablas involucradas		campos	tipo/largo
	v_fact_ordenes_compra	compra	id_compra	int
		usuarios	id_usuario_comprador	int
		infousuarios	fechaCompra	date
		tipousuario	totalUSD	int
		ciudad	nombreComprador	varchar(61)
		pais	nombreUsuario	varchar(30)
			email	varchar(80)
			tipoUsuario	varchar(45)
			telefono	varchar(20)
			ciudad	varchar(45)
			pais	varchar(45)
			lat	decimal(10,7)
			lon	decimal(10,7)
			id_tipoUsuario	int
			id_ciudad	int
			id_pais	int

```
CREATE OR REPLACE VIEW v_fact_ordenes_compra AS
select
  c.id_compra,
  c.id_usuario_comprador,
  c.fechaCompra,
  c.totalUSD,
  concat(iu.nombre, " ", iu.apellido) as nombreComprador,
  u.nombreUsuario as nombreUsuario,
  u.email,
  tu.tipoUsuario,
  iu.telefono,
  ciu.ciudad,
  p.pais,
  ciu.lat,
  ciu.lon,
  u.id_tipoUsuario,
  ciu.id_ciudad,
  p.id_pais
from compra as c
inner join usuarios as u on c.id_usuario_comprador = u.id_usuario
inner join infousuarios as iu on iu.id_usuario = c.id_usuario_comprador
inner join tipousuario as tu on tu.id_tipoUsuario = u.id_tipoUsuario
inner join ciudad as ciu on ciu.id_ciudad = iu.id_ciudad
inner join pais as p on p.id_pais = ciu.id_pais
order by c.id_compra ASC
;
```

	id_compra	id_usuario_comprador	fechaCompra	totalUSD	nombreComprador	nombreUsuario	email	tipoUsuario	telefono	ciudad	pais	lat	lon	id_tipoUsuario	id_ciudad	id_pais
►	1	97	2023-02-15	89	Arlinda Ogborne	chefford2m	mmourgue2m@amazonaws.com	usuario	+62 (628) 997-4677	Talobeiras	Brasil	-15.8142935	-42.2273780	2	424	26
	2	126	2023-07-17	104	Casper Cerman	bmegahey3f	chailey3f@fema.gov	usuario	+86 (329) 527-6893	Patos	Brasil	-7.0269670	-37.2776539	2	430	26
	3	156	2023-07-28	71	Pennie Deerness	jmungham49	dncgavin49@desdev.cn	usuario	+63 (707) 734-4556	Ansermanuevo	Colombia	4.7703172	-76.0623979	2	701	42
	4	477	2023-08-19	28	Quintina Philippard	gsoaresd6	ciwolfendelid6@51.la	usuario	+998 (675) 580-8996	Salamanca	Chile	-31.7824930	-70.9607105	2	180	38
	5	386	2023-08-22	53	Paulie Statton	mfermsan	glatorrean@eventbrite.com	usuario	+62 (406) 992-4088	Zaragoza	Colombia	7.4868800	-74.8672519	2	670	42
	6	52	2023-09-28	39	Roxie Woodwin	sworshall1d	wduly1d@sis.edu	usuario	+245 (395) 838-7099	Seattle	Estados Unidos	47.6025001	-122.3385759	2	812	60
	7	313	2023-12-21	113	Blandh Khosa	sfittall8m	omulhall8m@theatlantic.com	usuario	+63 (365) 514-1118	Marechal Cândido Rondon	Brasil	-24.7073654	-54.0688562	2	409	26
	8	441	2024-01-22	91	Judon Braime	charrilic6	rdunderdalec6@wired.com	usuario	+1 (713) 349-0169	Guaratinguetá	Brasil	-22.8140057	-45.1904004	2	314	26
	9	457	2024-01-26	9	Moir Ventom	erobrosecm	jmuriecm@go.com	usuario	+62 (800) 828-3353	Gualedguay	Argentina	-34.6961915	-58.8423829	2	53	9

Vista que muestra información privada de tarjetas de crédito almacenadas

VISTA	tablas involucradas		campos	tipo/largo
	v_fact_tarjetas	tarjetas	id_tarjeta	int
		marcaTarjeta	marca	varchar(40)
			numero	varchar(15)
			vencimiento	int
			cvv	varchar(3)
			titular	varchar(50)
			documentoTitular	varchar(15)
			id_marcaTarjeta	int

```
CREATE OR REPLACE VIEW v_fact_tarjetas AS
select
  tarj.id_tarjeta,
  marca.marca,
  concat(left(tarj.numero,3), " *** ", right(tarj.numero,3) ) as numero,
  tarj.vencimiento,
  "****" as cvv,
  tarj.titular,
  tarj.documentoTitular,
  tarj.id_marcaTarjeta
from tarjetas as tarj
inner join marcaTarjeta as marca on tarj.id_marcaTarjeta = marca.id_marcaTarjeta
order by id_tarjeta
;
```

	id_tarjeta	marca	numero	vencimiento	cvv	titular	documentoTitular	id_marcaTarjeta
►	1	VISA	401 **** 881	202606	***	Adrian Wippermann	25660751	1
	2	UnionPay	355 **** 103	202502	***	Noll Romanet	29301501	6
	3	Diners Club	362 **** 193	202407	***	Sarena Shemmans	36595202	5
	4	UnionPay	358 **** 620	202701	***	Jennica Sheering	18991917	6
	5	UnionPay	353 **** 168	202505	***	Humberto Eve	23725764	6
	6	Amex	337 **** 632	202507	***	Tessil Jacobsohn	31059073	3
	7	Diners Club	367 **** 477	202501	***	Carmen Van Castele	33268135	5
	8	Maestro	503 **** 293	202606	***	Louisa Emslie	22816818	8
	9	Maestro	676 **** 991	202604	***	Ebba Ede	39900074	8
	10	VISA	356 **** 146	202506	***	Nat Nuttey	15176023	1
	11	Diners Club	360 **** 931	202504	***	Melonie Miguel	37617391	5

7 Vistas (c)

Vista que muestra las fotografías que tienen comentarios, con su álbum de pertenencia, usuario comentador, fecha.

VISTA			
v_fact_fotos_comentadas	tablas involucradas	campos	tipo/largo
	fotografias	id_fotografia	int
	comentarios	nombreAlbum	varchar(50)
	usuarios	comentario	text
	albumes	usuarioComentador	varchar(30)
		fechaCargaFoto	date
		fechaComentario	date
		id_usuario_comentador	int
		id_album	int

id_fotografia	nombreAlbum	comentario	usuarioComentador	fechaCargaFoto	fechaComentario	id_usuario_comentador	id_album
4606	Latidos de la Naturaleza	La atmósfera que creaste en esta imagen es ev...	jdormonr	2023-02-17	2023-02-22	30	232
3519	Horizontes en Armonía	La calidad de la foto es excelente, se aprecian ...	cmeineck6o	2023-02-05	2023-04-18	243	178
768	Instantes Eternos	Me gusta la naturalidad de esta foto, se siente ...	arigneyar	2023-05-04	2023-05-06	390	40
1041	Instantes de Magia	La edición es muy suave y realiza la belleza natu...	sdaens2t	2023-02-21	2023-05-14	104	53
550	Horizontes Infinitos	La calidad de la foto es genial, se ve muy nítida ...	tpolle9s	2023-02-03	2023-05-15	355	29
184	Tesoros de la Naturaleza	La edición sutil realiza la belleza natural de la ima...	jgarett9f	2023-04-17	2023-05-20	342	10
2361	Instantes de Libertad	Los colores pastel le dan un toque suave y her...	tdelaperrelleau	2023-04-13	2023-06-10	393	119

```
CREATE OR REPLACE VIEW v_fact_fotos_comentadas AS
SELECT
  f.id_foto AS id_fotografia,
  alb.nombre AS nombreAlbum,
  com.comentario,
  us.nombreUsuario AS usuarioComentador,
  f.fechaCarga AS fechaCargaFoto,
  com.fecha AS fechaComentario,
  com.id_usuario AS id_usuario_comentador,
  f.id_album
FROM fotografias AS f
INNER JOIN comentarios AS com ON f.id_foto = com.id_foto
INNER JOIN usuarios AS us ON us.id_usuario = com.id_usuario
INNER JOIN albumes AS alb ON alb.id_album = f.id_album
WHERE EXISTS(
  SELECT DISTINCT(id_foto)
  FROM comentarios AS c
  WHERE c.id_foto = f.id_foto
)
ORDER BY fechaComentario ASC
;
```

Vista de control, que muestra aquellos registros de pagos que no han sido aprobados o con errores, cuyas órdenes de compra no tiene un importe asociado por ejemplo, por lo general son registros de pagos que tienen un estado no correcto.

VISTA			
v_fact_pagos_no_procesados	tablas involucradas	campos	tipo/largo
	pagos	id_pago	int
	compra	fechaHora	timestamp
	estadopago	id_compra	int
	tarjetas	fechaCompra	date
	marcatarjeta	totalUSD	int
	usuarios	tarjeta	varchar(40)
		usuarioComprador	varchar(30)
		estadoPago	varchar(30)
		id_estadoPago	int
		id_tarjeta	int
		id_usuario_comprador	int

	id_pago	fechaHora	id_compra	fechaCompra	totalUSD	tarjeta	usuarioComprador	estadoPago	id_estadoPago	id_tarjeta	id_usuario_comprador
▶	13	2024-02-20 03:29:00	13	2024-02-20	NULL	MasterCard	bhendrikse9p	error	4	62	352
	21	2024-03-27 08:24:00	21	2024-03-27	NULL	Diners Club	ccrinkley4x	error	4	11	180
	52	2024-04-24 19:57:00	52	2024-04-24	NULL	MasterCard	woxbe64	error	4	72	223

```
CREATE OR REPLACE VIEW v_fact_pagos_no_procesados AS
SELECT
  p.id_pago,
  p.fechaHora,
  p.id_compra,
  c.fechaCompra,
  c.totalUSD,
  mt.marca AS tarjeta,
  u.nombreUsuario AS usuarioComprador,
  e.estado as estadoPago,
  p.id_estadoPago,
  p.id_tarjeta,
  c.id_usuario_comprador
FROM pagos AS p
INNER JOIN compra AS c ON p.id_compra = c.id_compra
INNER JOIN estadopago AS e ON e.id_estadoPago = p.id_estadoPago
INNER JOIN tarjetas AS t ON p.id_tarjeta = t.id_tarjeta
INNER JOIN marcatarjeta AS mt ON mt.id_marcaTarjeta = t.id_marcaTarjeta
INNER JOIN usuarios AS u ON u.id_usuario = c.id_usuario_comprador
WHERE e.id_estadoPago != 1
ORDER BY p.id_pago
;
```


7 Vistas (d)

Vista que muestra información relacionada a los álbumes, sus vendedores (fotógrafos) y sus usuarios compradores así como la fecha de la operación de compra/venta

VISTA	VISTA		
	tablas involucradas	campos	tipo/largo
	álbumes	id_album	int
	itemscompra	nombreAlbum	varchar(50)
	compra	id_usuario_fotografo	int
		id_usuario_comprador	int
		nombreFotografo	varchar(60)
		nombreUsuarioComprador	varchar(60)
		importeUSD	int
		fecha_Compra_Venta	date

```
CREATE OR REPLACE VIEW v_fact_album_compra_venta AS
SELECT
  a.id_album,
  a.nombre AS nombreAlbum,
  a.id_usuario AS id_usuario_fotografo,
  c.id_usuario_comprador,
  f_getUser_nombre(a.id_usuario) AS nombreFotografo,
  f_getUser_nombre(c.id_usuario_comprador) AS nombreUsuarioComprador,
  f_precio_album(a.id_album) AS importeUSD,
  c.fechaCompra AS fecha_Compra_Venta
FROM albumes AS a
INNER JOIN itemscompra AS ic ON ic.id_album = a.id_album
INNER JOIN compra AS c ON c.id_compra = ic.id_compra
;
```

	id_album	nombreAlbum	id_usuario_fotografo	id_usuario_comprador	nombreFotografo	nombreUsuarioComprador	importeUSD	fecha_Compra_Venta
▶	110	Encuentros del Camino	13	97	Willard Chrystal	Arlinda Ogborne	28	2023-02-15
	187	Encuentros con la Luz	424	97	Glyn Luckman	Arlinda Ogborne	19	2023-02-15
	172	Espacios de Descanso	396	97	Marjory Kimmings	Arlinda Ogborne	19	2023-02-15
	227	Espacios de Serenidad	398	97	Amos Emilien	Arlinda Ogborne	23	2023-02-15
	58	Belleza en Detalles	256	126	Clemmie Duffit	Casper Cerman	18	2023-07-17
	90	Horizontes en Armonía	20	126	Minda Geeves	Casper Cerman	24	2023-07-17
	166	Latidos del Mundo	389	126	Vi Reynolds	Casper Cerman	9	2023-07-17
	194	Espacios de Libertad	339	126	Chrisse Casine	Casper Cerman	23	2023-07-17
	134	Horizontes en Armonía	396	126	Marjory Kimmings	Casper Cerman	30	2023-07-17
	133	Latidos del Corazón	415	156	Orbadiah Webland	Pennie Deerness	11	2023-07-28
	53	Instantes de Magia	254	156	Francois Sprigings	Pennie Deerness	15	2023-07-28
	37	Emociones Capturadas	22	156	Christine Limmer	Pennie Deerness	23	2023-07-28
	35	Susurros del Viento	344	156	Wendye Davidovsky	Pennie Deerness	22	2023-07-28
	44	Reflejos del Alma	350	477	Celine Rudham	Quintina Philippard	28	2023-08-19
	99	Encuentros con la Luz	316	386	Noami Harrill	Paulie Statton	30	2023-08-22
	248	Miradas del Pasado	424	386	Glyn Luckman	Paulie Statton	23	2023-08-22
	152	Instantes de Magia	272	52	Grantley Hawtin	Roxie Woodwin	28	2023-09-28
	22	Latidos de la Ciudad	349	52	Silvan Arni	Roxie Woodwin	11	2023-09-28

FUNCIONES

8 Funciones (a)

FUNCION				
f_precio_album	tipo funcion	tablas involucradas	parametros	retorna
	READS SQL DATA	precioalbum	p_id_album	INT
	Funcion que devuelve el precio más reciente de un álbum dado			

```

26
27 • select f_precio_album(15);

```

Result Grid | Filter Rows: | Exp

f_precio_album(15)
29

```

DELIMITER $$
CREATE FUNCTION f_precio_album (p_id_album INT) RETURNS INT
READS SQL DATA
BEGIN
    DECLARE cantidad INT default 0;
    DECLARE precio INT DEFAULT 0;

    SELECT COUNT(*) INTO cantidad
    FROM precioalbum
    WHERE id_album=(p_id_album)
    GROUP BY id_album;

    IF cantidad > 0 THEN
        SELECT importeUSD INTO precio
        FROM precioAlbum
        WHERE id_album IN (p_id_album)
        ORDER BY fechaAltaPrecio DESC
        LIMIT 1;
    END IF;
    RETURN precio;
END$$

```

FUNCION				
f_total_orden_compra	tipo funcion	tablas involucradas	parametros	retorna
	READS SQL DATA	itemscompra	p_id_orden_compra	INT
	Función que suma el importe de venta vigente de todos los elementos que conforman una orden de compra usado para calcular el importe total de las ordenes de compra			

```

26
27 • select f_total_orden_compra(47);

```

Result Grid | Filter Rows: | Exp

f_total_orden_compra(47)
62

```

DELIMITER $$
CREATE FUNCTION f_total_orden_compra (p_id_orden_compra INT) returns INT
READS SQL DATA
BEGIN
    DECLARE total INT DEFAULT 0;

    SELECT SUM( f_precio_album(id_album) ) INTO total
    FROM itemscompra
    WHERE id_compra = p_id_orden_compra;

    RETURN total;
END $$
DELIMITER ;

```

8 Funciones (b)

FUNCION				
	tipo funcion	tablas involucradas	parametros	retorna
f_max_comentarios_foto	READS SQL DATA	comentarios		INT
	Función que devuelve un escalar que representa la máxima cantidad de comentarios que existe en una misma fotografía			

27 • `select f_max_comentarios_foto();`

<

Result Grid ☐ Filter Rows: Export

	f_max_comentarios_foto()
▶	2

```

DELIMITER $$
CREATE FUNCTION f_max_comentarios_foto() RETURNS INT
READS SQL DATA
BEGIN
    DECLARE cantidad INT;

    SELECT count(*) AS max_comentarios_foto
        INTO cantidad
    FROM comentarios
    GROUP BY id_foto
    ORDER BY 1 DESC
    LIMIT 1;

    RETURN cantidad;
END $$
DELIMITER ;

```

FUNCION				
	tipo funcion	tablas involucradas	parametros	retorna
f_max_comentarios_usuario	READS SQL DATA	comentarios		INT
	función que devuelve un escalar que representa la cantidad máxima de comentarios realizados por un mismo usuario			

27 • `select f_max_comentarios_usuario();`

<

Result Grid ☐ Filter Rows: Export

	f_max_comentarios_usuario()
▶	5

```

DELIMITER $$
CREATE FUNCTION f_max_comentarios_usuario() RETURNS INT
READS SQL DATA
BEGIN
    DECLARE cantidad INT;

    select count(*) as max_comentarios_usuario
        INTO cantidad
    from comentarios
    group by id_usuario
    order by 1 desc
    limit 1;

    RETURN cantidad;
END $$
DELIMITER ;

```

8 Funciones (c)

FUNCION				
f_cant_vendidos_album	tipo funcion	tablas involucradas	parametros	retorna
	READS SQL DATA	albumes	p_id_album	INT
		itemscompra		
	Función que devuelve la cantidad de veces que se vendió un album dado su id_album			

14 • **SELECT** f_cant_vendidos_album(114);

Result Grid | Filter Rows: | Exp

f_cant_vendidos_album(114)
3

```
DELIMITER $$
CREATE FUNCTION f_cant_vendidos_album(p_id_album INT) RETURNS INT
READS SQL DATA
BEGIN
    DECLARE cant INT;

    SELECT count(*)
        INTO cant
    FROM albumes AS a
    INNER JOIN itemscompra AS ic ON a.id_album = ic.id_album
    WHERE a.id_album = p_id_album;

    RETURN cant;

END $$
DELIMITER ;
```

FUNCION				
f_getUser_nombre	tipo funcion	tablas involucradas	parametros	retorna
	READS SQL DATA	infousuarios	p_id_usuario	VARCHAR(60)
	Función que devuelve el nombre y apellido de un usuario dado su id_usuario esta función se utiliza en la vista v_album_compra_venta, para evitar relacionar 2 tablas (una de comprador y otra de vendedor) a la misma tabla 'infousuarios' mediante 2 joins a la misma tabla.			

16 • **SELECT** f_getUser_nombre(1);

Result Grid | Filter Rows: | Exp

f_getUser_nombre(1)
Jose Soria

```
DELIMITER $$
CREATE FUNCTION f_getUser_nombre(p_id_usuario INT) RETURNS VARCHAR(60)
READS SQL DATA
BEGIN
    RETURN (
        SELECT concat(iu.nombre, " ", iu.apellido)
        FROM infousuarios AS iu
        WHERE iu.id_usuario = p_id_usuario
    );

END $$
DELIMITER ;
```

STORED PROCEDURES

9 Stored Procedures (a)

PROCEDIMIENTO			
	tablas involucradas	tipo parametro	parametros
p_elimina_comentarios	comentarios	IN	p_id_album
	Procedimiento que elimina todos los comentarios para todas las fotografias incluidas en un album dado como parámetro		
	Ejemplo: CALL p_elimina_comentarios(29);		

PROCEDIMIENTO			
	tablas involucradas	tipo parametro	parametros
p_muestra_elimina_comentarios	comentarios	IN	p_id_album
	Procedimiento que muestra los registros a eliminar si se utilizara el proced. anterior "p_elimina_comentarios", antes de proceder a su ejecución		

```
DELIMITER $$
CREATE PROCEDURE p_elimina_comentarios(IN p_id_album INT)
BEGIN

    DELETE FROM comentarios
    WHERE id_foto IN (
        SELECT id_foto from fotografias
        WHERE id_album = p_id_album
    );

END $$
DELIMITER ;
```

5 • `SELECT * FROM v_fotos_comentadas WHERE id_album = 80;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

id_fotografia	nombreAlbum	comentario	usuarioComentador	fechaCargaFoto	fechaComentario	id_usuario_comentador	id_album
1582	Belleza en Silencio	La iluminación en esta imagen es perfecta, resal...	cdroganix	2023-07-27	2023-07-31	72	80
1577	Belleza en Silencio						
1581	Belleza en Silencio						
1569	Belleza en Silencio						
1576	Belleza en Silencio						
1574	Belleza en Silencio						

6 • `CALL p_elimina_comentarios(80);`

6 • `SELECT * FROM v_fotos_comentadas WHERE id_album = 80;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

id_fotografia	nombreAlbum	comentario	usuarioComentador	fechaCargaFoto	fechaComentario	id_usuario_comentador	id_album
---------------	-------------	------------	-------------------	----------------	-----------------	-----------------------	----------

24 • `CALL p_muestra_elimina_comentarios(32);`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

id_comentario	fecha	comentario	id_usuario	id_foto
257	2024-02-21	La calidad de la foto es excelente, se aprecian ...	331	598
202	2024-01-27	La edición sutil resalta la belleza natural de la es...	159	608

```
DELIMITER $$
CREATE PROCEDURE p_muestra_elimina_comentarios(IN p_id_album INT)
BEGIN

    SELECT * FROM comentarios
    WHERE id_foto IN (
        SELECT id_foto from fotografias
        WHERE id_album = p_id_album
    );

END $$
DELIMITER ;
```

9 Stored Procedures (b)

PROCEDIMIENTO	tablas involucradas	tipo parametro	parametros
p_crear_usuario	usuarios	IN	x_email
		IN	x_contraseña
		IN	x_nombreUsuario
		IN	x_id_tipoUsuario
Procedimiento para la creación de un usuario nuevo en la base			

```

22 CALL p_crear_usuario( 'usuario.de.prueba@yopmail.com',
23                        'hU84_33,a=km2J$#!z_',
24                        'test_user_891',
25                        4);
    
```

id_usuario	email	contraseña	nombreUsuario	id_tipoUsuario
500	usuario.de.prueba@yopmail.com	hU84_33,a=km2J\$#!z_	test_user_891	4
NULL	NULL	NULL	NULL	NULL

PROCEDIMIENTO	tablas involucradas	tipo parametro	parametros
p_rellena_importe_ordenes	compra		
Procedimiento que rellena los valores de la columna 'totalUSD' de la traba compra'. Dicho campo almacena los importes totales de una orden de compra, se suman los importes de cada elemento que conforma la orden de compra.			
Se hace uso de la función: f_total_orden_compra			

	id_compra	id_usuario_comprador	fechaCompra	totalUSD
1	97		2023-02-15	89
2	126		2023-07-17	104
3	156		2023-07-28	71
4	477		2023-08-19	28
5	386		2023-08-22	53
6	52		2023-09-28	39
7	313		2023-12-21	113

	id_compra	id_usuario_comprador	fechaCompra	totalUSD
1	97		2023-02-15	89
2	126		2023-07-17	104
3	156		2023-07-28	71
4	477		2023-08-19	28
5	386		2023-08-22	53
6	52		2023-09-28	39
7	313		2023-12-21	113

```

DELIMITER $$
CREATE PROCEDURE p_crear_usuario(
    IN x_email VARCHAR(80),
    IN x_contraseña VARCHAR(50),
    IN x_nombreUsuario VARCHAR(30),
    IN x_id_tipoUsuario INT
)
BEGIN
    INSERT INTO usuarios (email, contraseña, nombreUsuario, id_tipoUsuario)
    VALUES (x_email, x_contraseña, x_nombreUsuario, x_id_tipoUsuario);
END $$
DELIMITER ;
    
```

```

DELIMITER $$
CREATE PROCEDURE p_rellena_importe_ordenes()
BEGIN
    UPDATE compra
    SET totalUSD = f_total_orden_compra(compra.id_compra)
    WHERE totalUSD IS NULL;
END $$
DELIMITER ;
    
```


9 Stored Procedures (c)

Procedimiento que genera tablas espejo, que provienen de las vistas ya creadas en el apartado anterior, para que funcionen como tablas de consulta en las herramientas de BI y no afectar el funcionamiento de las tablas de producción. Estas tablas espejo se eliminan y se crean en cada llamado al procedimiento, y se guardarán con datos entre las fechas indicadas en parámetros, para poder restringir el volumen del análisis si la base de datos cuenta con mucha info que no sea necesario analizar.

Estas tablas no tienen código de definición en el archivo con Scripts de creación de la estructura DDL, sino que por el contrario, son generadas a través de un SELECT de cada una de las Vistas involucradas.

Vistas usadas:

v_fact_album_compra_venta
v_fact_fotos_comentadas
v_fact_ordenes_compra
v_fact_ordenes_compra_detalle
v_fact_usuarios_detalle

Tablas generadas por el procedure:

z_esp_album_compra_venta
z_esp_fotos_comentadas
z_esp_ordenes_compra
z_esp_ordenes_compra_detalle
z_esp_usuarios_detalle

PROCEDIMIENTO			
p_genera_tablas_analisis	tablas involucradas	tipo parametro	parametros
	z_esp_album_compra_venta	IN	fechaDesde
	z_esp_fotos_comentadas	IN	fechaHasta
	z_esp_ordenes_compra		
	z_esp_ordenes_compra_detalle		
	z_esp_usuarios_detalle		

```

30 • call p_genera_tablas_analisis('2023-01-01', '2024-05-31');
31
32 • SELECT * FROM z_esp_album_compra_venta;
33 • SELECT * FROM z_esp_fotos_comentadas;
34 • SELECT * FROM z_esp_ordenes_compra;
35 • SELECT * FROM z_esp_ordenes_compra_detalle;
36 • SELECT * FROM z_esp_usuarios_detalle;

```

id_usuario	tipoUsuario	nombreUsuario	email	nombre	apellido	fechaCreacion	fechaNac	telefono	ciudad	pais	id_infoUsuarios	ik
2	admin	weigandtm	mabel.weigandt@photo.com	Mabel	Weigandt	2023-03-13	1980-09-09	+54 (223) 377-7093	Mar del Plata	Argentina	77	1
1	admin	equinox	jose.soria@photo.com	Jose	Soria	2023-01-01	1981-01-06	+54 (223) 170-2332	Mar del Plata	Argentina	1	1
28	usuario	lprivettp	cfrandip@cornell.edu	Mildred	Grahamslaw	2024-05-03	1996-11-02	+351 (768) 637-7433	Chinú	Colombia	499	2
395	usuario	sstoakesaw	nlaboraaa@accuweather.com	Vivia	Rouke	2024-04-29	2005-04-25	+7 (947) 718-0981	Belén	Uruguay	497	2
297	usuario	pdillway86	kfeveryear86@infoseek.co.jp	Kissiah	Dilks	2024-04-29	1989-07-02	+420 (876) 166-3292	Villa Dolores	Argentina	496	2
341	usuario	rhavle9a	nikitau9a@theguardian.com	Shobhana	Wicker	2024-04-28	2000-05-22	+86 (106) 333-1977	Greenboro	Estados Unidos	405	2

z_esp_album_compra_venta 1 z_esp_fotos_comentadas 2 z_esp_ordenes_compra 3 z_esp_ordenes_compra_detalle... z_esp_usuarios_detalle 5 x

```

DELIMITER $$
CREATE PROCEDURE p_genera_tablas_analisis(IN fechaDesde DATE, IN fechaHasta DATE)
BEGIN
    DROP TABLE IF EXISTS z_esp_album_compra_venta;
    CREATE TABLE z_esp_album_compra_venta AS SELECT * FROM v_fact_album_compra_venta WHERE fecha_Compra_Venta BETWEEN fechaDesde AND fechaHasta;

    DROP TABLE IF EXISTS z_esp_fotos_comentadas;
    CREATE TABLE z_esp_fotos_comentadas AS SELECT * FROM v_fact_fotos_comentadas WHERE fechaComentario BETWEEN fechaDesde AND fechaHasta;

    DROP TABLE IF EXISTS z_esp_ordenes_compra;
    CREATE TABLE z_esp_ordenes_compra AS SELECT * FROM v_fact_ordenes_compra WHERE fechaCompra BETWEEN fechaDesde AND fechaHasta;

    DROP TABLE IF EXISTS z_esp_ordenes_compra_detalle;
    CREATE TABLE z_esp_ordenes_compra_detalle AS SELECT * FROM v_fact_ordenes_compra_detalle WHERE fechaCompra BETWEEN fechaDesde AND fechaHasta;

    DROP TABLE IF EXISTS z_esp_usuarios_detalle;
    CREATE TABLE z_esp_usuarios_detalle AS SELECT * FROM v_fact_usuarios_detalle WHERE fechaCreacion BETWEEN fechaDesde AND fechaHasta;

END $$
DELIMITER ;

```

TRIGGERS

10 Triggers

TRIGGER			
	tipo trigger	tablas involucradas	propósito
t_audit_usuarios_insert	AFTER INSERT	usuarios	auditoría
t_audit_usuarios_update	AFTER UPDATE	usuarios	auditoría
t_audit_usuarios_delete	AFTER DELETE	usuarios	auditoría
Triggers de auditoría con la finalidad de reflejar modificaciones en la tabla 'usuarios'			

34 • `SELECT * FROM usuarios_audit;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: `FA`

	id_audit	tipo	fecha	id_usuario	email	contraseña	nombreUsuario	id_tipoUsuario
▶	1	INSERT_NEW	2024-05-11 02:24:26	505	tester@test.com.ar	sda#29d9s<!B,?	usuario_tester	3
	2	UPDATE_OLD	2024-05-11 02:28:30	505	tester@test.com.ar	sda#29d9s<!B,?	usuario_tester	3
	3	UPDATE_NEW	2024-05-11 02:28:30	505	tester@test.com.ar	DUF"#)Dxows_s	usuario_tester	3
	4	DELETE_OLD	2024-05-11 02:30:21	505	tester@test.com.ar	DUF"#)Dxows_s	usuario_tester	3
	5	UPDATE_OLD	2024-05-11 02:44:52	503	TEST@TEST.COM	asd882kd_AS"35_	prueba	3
	6	UPDATE_NEW	2024-05-11 02:44:52	503	TEST@TEST.COM	asd882kd_AS"35_	prueba	4
	7	DELETE_OLD	2024-05-11 02:45:24	503	TEST@TEST.COM	asd882kd_AS"35_	prueba	4
	8	INSERT_NEW	2024-05-13 17:02:05	500	usuario.de.prueba@yopmail.com	hU84_33,a=km2J\$#!z_	test_user_891	4
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

DELIMITER $$
CREATE TRIGGER t_audit_usuarios_insert AFTER INSERT ON usuarios FOR EACH ROW
BEGIN
    INSERT INTO usuarios_audit(tipo, fecha, id_usuario, email, contraseña, nombreUsuario, id_tipoUsuario)
    VALUES('INSERT_NEW', SYSDATE(), NEW.id_usuario, NEW.email, NEW.contraseña, NEW.nombreUsuario, NEW.id_tipoUsuario);
END $$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER t_audit_usuarios_update AFTER UPDATE ON usuarios FOR EACH ROW
BEGIN
    INSERT INTO usuarios_audit(tipo, fecha, id_usuario, email, contraseña, nombreUsuario, id_tipoUsuario)
    VALUES('UPDATE_OLD', SYSDATE(), OLD.id_usuario, OLD.email, OLD.contraseña, OLD.nombreUsuario, OLD.id_tipoUsuario);

    INSERT INTO usuarios_audit(tipo, fecha, id_usuario, email, contraseña, nombreUsuario, id_tipoUsuario)
    VALUES('UPDATE_NEW', SYSDATE(), NEW.id_usuario, NEW.email, NEW.contraseña, NEW.nombreUsuario, NEW.id_tipoUsuario);
END $$
DELIMITER ;

DELIMITER $$
CREATE TRIGGER t_audit_usuarios_delete AFTER DELETE ON usuarios FOR EACH ROW
BEGIN
    INSERT INTO usuarios_audit(tipo, fecha, id_usuario, email, contraseña, nombreUsuario, id_tipoUsuario)
    VALUES('DELETE_OLD', SYSDATE(), OLD.id_usuario, OLD.email, OLD.contraseña, OLD.nombreUsuario, OLD.id_tipoUsuario);
END $$
DELIMITER ;

```

INSERTS

11 Inserción de datos + SCRIPT

- La base de datos cuenta en todo su contenido con datos ficticios, generados mayoritariamente en la plataforma mockaroo y ChatGPT.

- Se utilizó Excel para la validación y corrección de errores en la consistencia de los datos.

A modo de ejemplo usando mockaroo y cargando un dataset con las ForeignKeys a utilizar en forma random, se daba la situación en la que una orden de compra era generada en una fecha "anterior" al alta del usuario que generaba dicha orden. O, en otro de los casos, que una compra era realizada en una fecha donde el/los álbumes comprados habían sido creados en una fecha posterior a la compra y varios casos similares.

- Finalmente desarrollé como apoyo un pequeño script en Visal Basic que generaba como resultado cadenas strings con todos los campos separados por comas y formando tuplas, respetando la ubicación de los campos a ser insertados, y obtener un resultado como el siguiente:

```
INSERT INTO compra (id_compra, id_usuario_comprador, fechaCompra, totalUSD)
VALUES
(1,97,'2023-02-15',NULL),
(2,126,'2023-07-17',NULL),
(3,156,'2023-07-28',NULL),
(4,477,'2023-08-19',NULL),
...
```

- De esta manera se cuenta con 2 scripts, uno para la creación de objetos y otro para insertar datos, que de esta manera al ejecutarlos generarán la estructura completa de la base, sus PK, FK y relaciones, la inserción de datos, y la creación del resto de los objetos: vistas, funciones, stored procedures y triggers.
- Utilice el link de descarga para acceder al contenido del proyecto, modelo DER y script [{descargar}](#)

TESTING

12 Testing

- Desde luego se puede probar cualquier parte de la base de datos. Dejo apuntado los Selects como punto de partida inicial, a los objetos creados:

Se deja en el proyecto el script “SELECTS para punto de partida.sql” con los siguientes:

```
-- Tablas
SELECT * FROM pais;
SELECT * FROM ciudad;
SELECT * FROM tipousuario;
SELECT * FROM usuarios;
SELECT * FROM infousuarios;

SELECT * FROM tipocontenido;
SELECT * FROM albumes;
SELECT * FROM fotografias;
SELECT * FROM precioalbum;
SELECT * FROM comentarios;

SELECT * FROM compra;
SELECT * FROM itemscompra;

SELECT * FROM marcatarjeta;
SELECT * FROM estadopago;
SELECT * FROM tarjetas;
SELECT * FROM pagos;

-- Vistas
SELECT * FROM v_fact_fotos_comentadas;
SELECT * FROM v_fact_ordenes_compra;
SELECT * FROM v_fact_ordenes_compra_detalle;
SELECT * FROM v_fact_pagos_no_procesados;
SELECT * FROM v_tarjetas;
SELECT * FROM v_fact_usuarios_detalle;
SELECT * FROM v_fact_album_compra_venta;

-- Funciones
SELECT f_max_comentarios_foto();
SELECT f_max_comentarios_usuario();
SELECT f_precio_album(1);
SELECT f_total_orden_compra(1);
SELECT f_cant_vendidos_album(114);
SELECT f_getUser_nombre(1);

-- Procedimientos
CALL p_crear_usuario('email.test@gamail.com', 'sdfj#"9sdd#d_xx', 'user000', '3');
CALL p_muestra_elimina_comentarios(151);
CALL p_elimina_comentarios(151);
CALL p_rellena_importe_ordenes();

-- Tabla auditoría Usuarios
SELECT * FROM usuarios_audit;

-- Tablas espejo para BI
CALL p_genera_tablas_analisis('2023-01-01', '2024-05-31');
SELECT * FROM z_esp_album_compra_venta;
SELECT * FROM z_esp_fotos_comentadas;
SELECT * FROM z_esp_ordenes_compra;
SELECT * FROM z_esp_ordenes_compra_detalle;
SELECT * FROM z_esp_usuarios_detalle;
```

ANALÍTICA

13 Analítica de datos

- Con la base de datos ya en funcionamiento, con sus datos ingestados y el resto de sus objetos generados, se procede a generar un reporte que pueda dar información visual sobre algunos de los aspectos que la base de datos contiene.
- Se elije hacer un reporte que se basa en información sobre los resultados de las compras realizada por usuarios, o bien de las ventas de los álbumes de fotografías que han hecho publicaciones en sus cuentas, en el período comprendido entre enero 2023 hasta mayo 2024.
- En su diseño visual, el reporte se basa en el minimalismo de elementos visuales y dejar solamente los imprescindibles para que cada elemento gráfico pueda ser interpretado por sí mismo sin mayor explicación. Como elemento que da interactividad se utiliza 1 solo filtro desplegable donde se podrá marcar un período anual, o mensual, o también selección múltiple si así se desea.
- El reporte cuenta con una barra descriptiva con KPIs globales y representativos, y en el apartado de elementos visuales, los gráficos que profundizan el detalle de los KPIs. Se habla principalmente de:

```
Evolución de ventas (en U$D)
Usuarios según mercado
Estacionalidad en Interacciones
Distribución de Estilos Fotográficos
Geo de volumen de compras
```

- Se hace uso de las tablas espejo destinadas para BI:

```
z_esp_album_compra_venta
z_esp_fotos_comentadas
z_esp_ordenes_compra
z_esp_ordenes_compra_detalle
z_esp_usuarios_detalle
```

A continuación una captura del reporte...



FOTÓGRAFO CON MÁS VENTAS

Grantley Hawtin

\$171

USUARIO CON MÁS COMPRAS

Bordy MacCaghan

\$136

ESTILO CON MÁS VENTAS

retratos

PRINCIPAL MERCADO POR VENTAS

Brasil

TOTAL VOLUMEN DE VENTAS

\$3.733

TOTAL COMISIONES

\$560

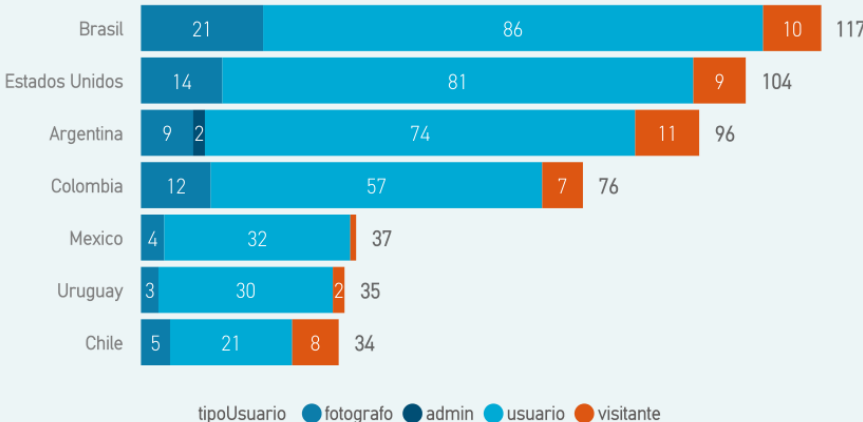
Evolución de ventas (en U\$D)

Evolución mensual en venta de álbumes fotográficos

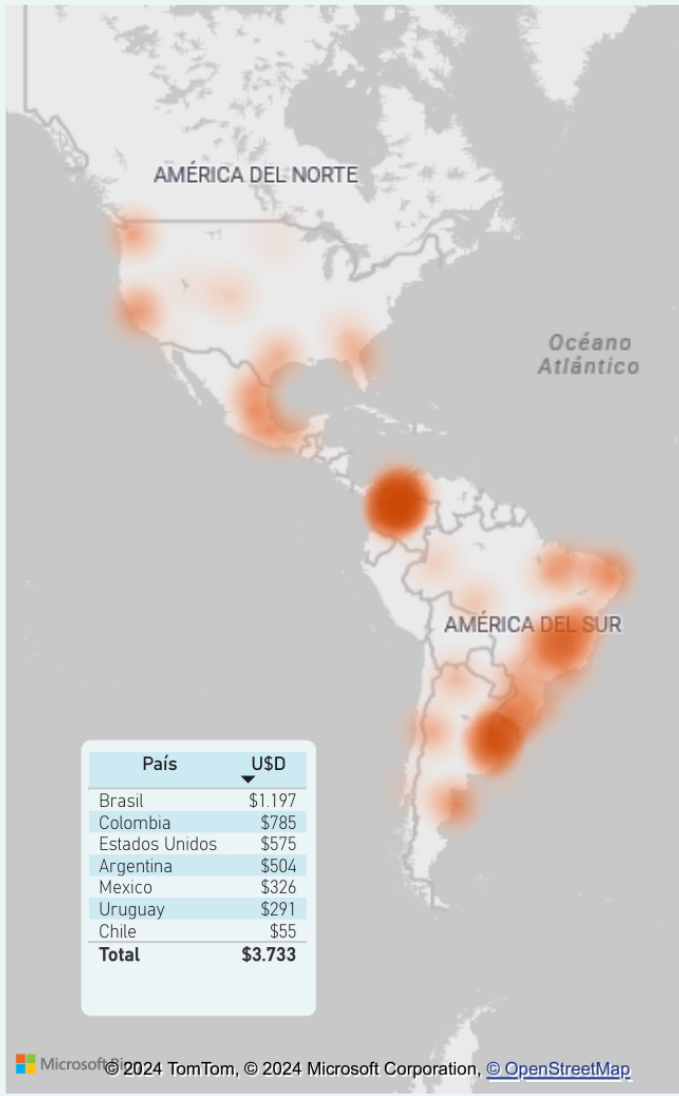


Usuarios según mercado

Composición de tipos de usuario disponibles por mercado

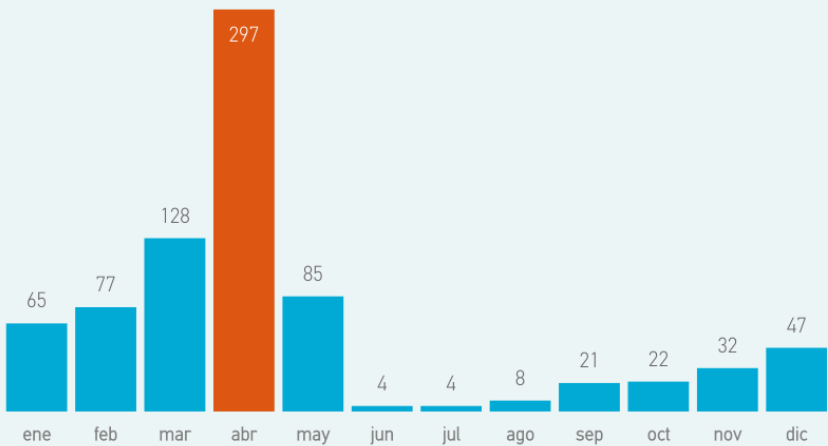


Geo de volumen de compras



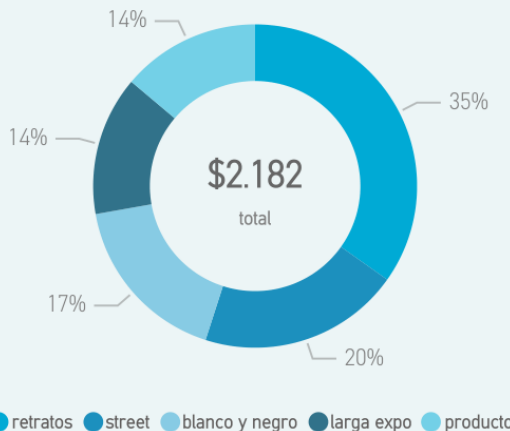
Estacionalidad en interacciones

Tendencia estacional de participación en período 2023 - 2024-mayo



Distribución de Estilos Fotográficos

TOP 5 de estilos fotográficos con mayor volumen de ventas



13 Conclusiones de analítica de datos

- Estas son las conclusiones principales que se obtienen del análisis del reporte:
 - Se puede advertir que los dos mercados con más usuarios son Brasil y Estados Unidos, sin embargo los dos mercados que más volumen de compras tienen son Brasil y Colombia.
 - En el tiempo el volumen de ventas fue bajo en sus comienzos, recién en el periodo marzo a mayo del 2024 se genera la mayor parte de las ventas.
 - El mes de abril es el mes que tiene el mayor pico de interactividad de sus usuarios, en términos estacionarios de periodo completo (2023-2024).
 - Los 3 estilos fotográficos más vendidos son Retratos, Street Photography, y Blanco y Negro, siendo Retratos el principal en los 2 períodos.
 - Principalmente usuarios del tipo "Usuario" son mucho más en cantidad que el tipo "Fotógrafos", lo que es bueno a la hora de generar compras pero siendo dependiente de que los usuarios del tipo "Fotógrafo" generen contenido para poner a la venta.

HERRAMIENTAS

14 Herramientas (a)

- Mockaroo para la generación de datos ficticios
- ChatGPT para la generación de datos ficticios
- MS Excel para la validación de los datos generados en Mockaroo
(aunque es una herramienta que no es deseable se tenga que usar para la gestión de una base de datos, se usó por practicidad y para resolver temas previos a la ingesta)
- Se utilizó PowerBI como herramienta de visualización
- MySQL gestor de base de datos la cual es el principal motivo de ésta presentación



14 Herramientas (b)

El proyecto en PowerBI se adjunta también como parte de la entrega.

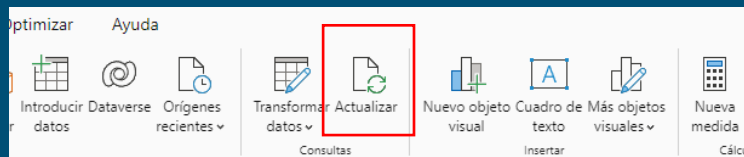
Para hacerlo correr, dado que no se incluye ningún dataset que alimente PowerBI, sino que se ejecuta mediante un conector .NET a la base de datos directamente, es necesario contemplar las siguientes pautas.



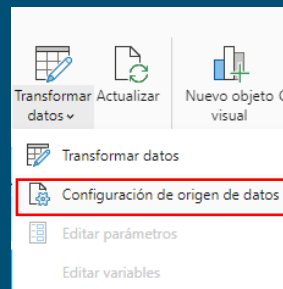
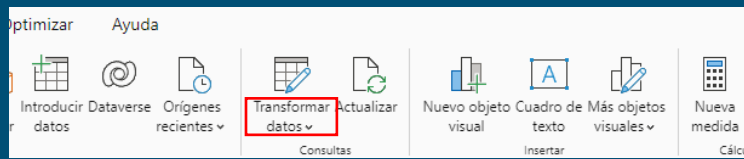
Si solamente va a consumirse el informe sin tener que acceder a la fuente de los datos, entonces abrir el documento **fotografía.pbix** directamente en PowerBI y usarlo sin más. La interactividad de sus elementos será funcional sin problemas.

Para lo siguiente, es necesario contar con el conector MySQL – PowerBI, para ello descargar Conector .NET para powerBI <https://dev.mysql.com/downloads/connector/net/>

Si se toca el botón Actualizar, avisará con un mensaje de error manteniendo la funcionalidad pero no se podrá acceder a la capa física ni hacer transformaciones a los datos desde el módulo PowerQuery.

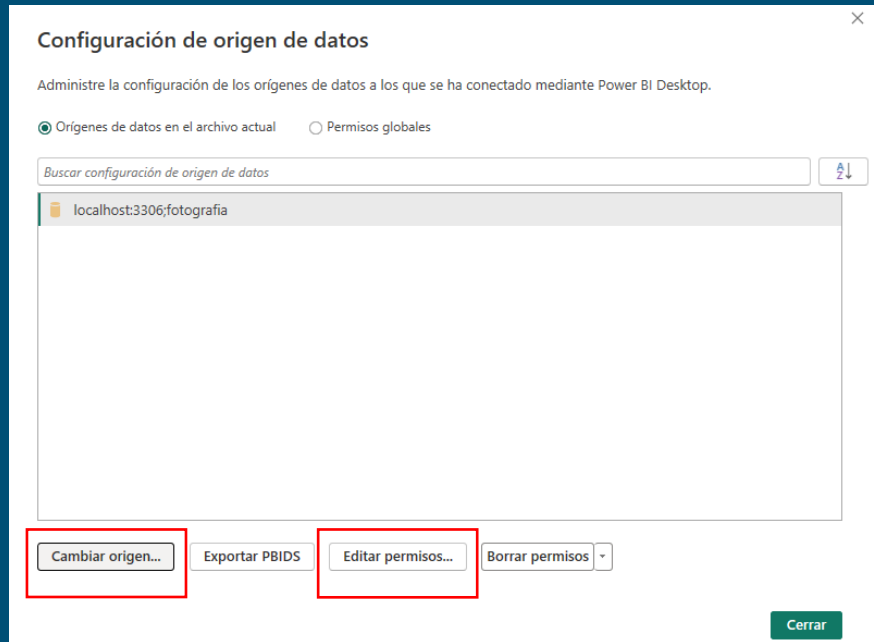


Para poder probar la conexión entre PowerBI y MySQL, acceder mediante la opción desplegable del botón Transformar datos, y luego “Configuración de origen de datos”



14 Herramientas (c)

En la siguiente pantalla tendremos que dirigirnos a los botones marcados debajo Cambiar Origen... y Editar permisos...



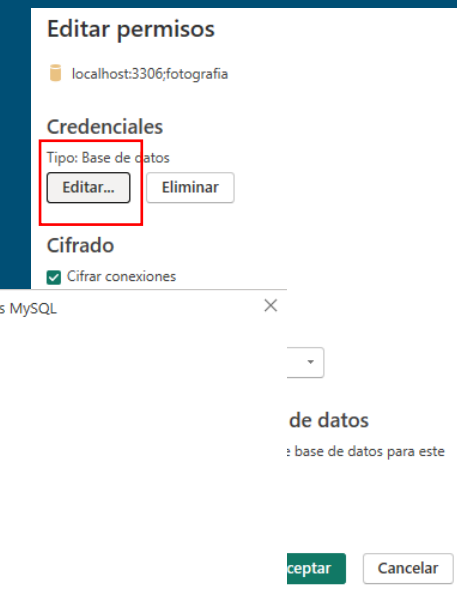
1-Cambiar origen...

Aquí establecer el servidor y puerto donde funciona el motor de MySQL al que haya ingestado el script SQL adjuntado en este trabajo. Indicarle el nombre del schema que será "fotografia"



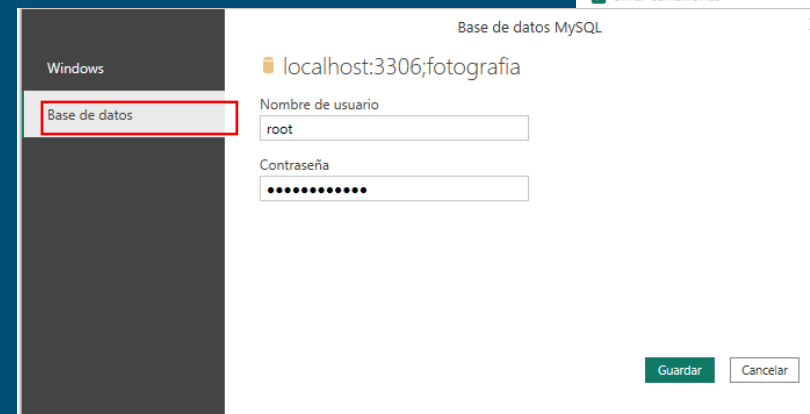
2-Editar permisos...

Aquí hay que ir al botón Editar... en el apartado Credenciales



Credenciales

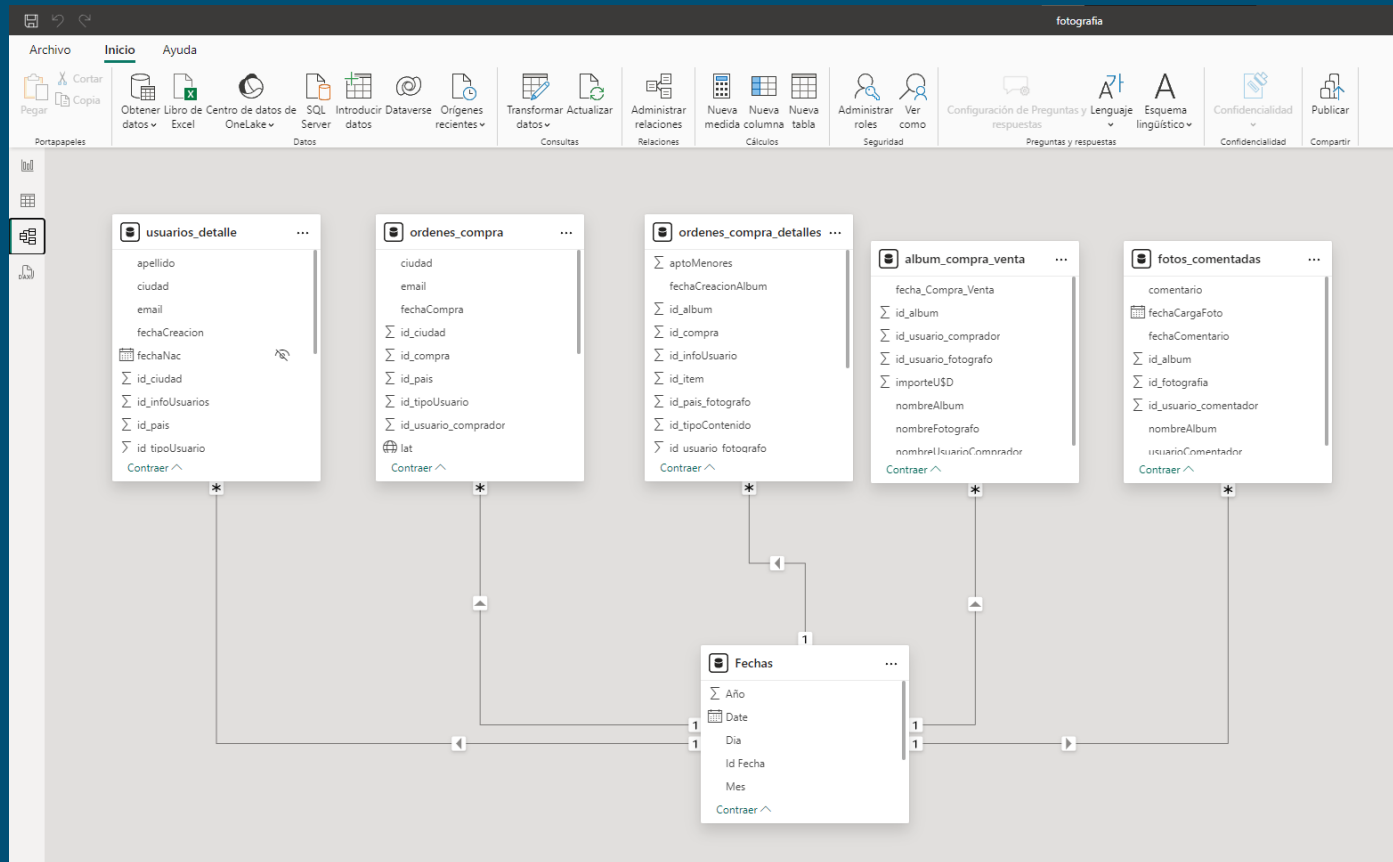
Por último en la pestaña "base de datos" ingresar las credenciales de la sesión en MySQL usada para consumir la base de datos. Aceptar en los cuadros de diálogo para confirmar.



Con esto, ya se puede Actualizar para testear la conexión y recorrer su capa física (módulo PowerQuery) en el caso de querer hacer transformaciones.

14 Herramientas (d)

Como parte de la capa lógica del informe se puede ver que aplica un modelo estrella entre la una tabla de fechas creada desde PowerBI y 5 tablas de hecho obtenidas de la conexión con MySQL, que son las vistas creadas para este fin ya mencionadas.



15 Futuras Líneas

Todo lo visto anteriormente es un enfoque iniciático para poner en funcionamiento el proyecto, sin embargo la idea es que se pueda ir escalando mucho más. Para ello será necesario seguir ampliando el modelo DER para las nuevas entidades que se vayan requiriendo.

A modo de ejemplo más opciones para la interacción de los usuarios (reacciones, compartir, etc...), como también disponibilizar diferentes monedas locales para el pago según la ubicación de cada usuario y estableciendo tablas internacionales de conversión de divisas.

Añadir más formas de pago, no solamente con tarjetas de crédito, sino también con débito locales, con entidades de pago internacionales como PayPal.

Establecer diferentes resoluciones de descarga de archivos.

Establecer abonos o membresías que en lugar que el usuario abone por el álbum que compre, tendría la posibilidad de abonar una membresía mensual de un bajo importe, con la posibilidad de hacer compras de álbumes pero con limitaciones en cantidades y estilos fotográficos.

PALABRAS FINALES

Aquí finaliza este recorrido por mi trabajo práctico poniendo a prueba la mayoría de los conocimientos y conceptos del curso. En el enlace compartido podrá encontrar todo el material necesario en este proyecto:

- Scripts SQL, con creación de todos los objetos + inserción de los datos
- Modelo DER, en formato .png y .mwb compatible con MySQL en el módulo Models
- Documento en PowerBI, dashboard con conexión a la base de datos
- El documento que se está leyendo, en formato .pdf y .pptx

[{descargar}](#)

Sin más que aportar, espero haber podido transmitir con claridad todos los puntos necesarios y que haya podido revisar el material arriba mencionado. Muchas gracias por el tiempo y conocimientos dedicados en la cursada!

alumno : **José Soria Díaz**
comisión : 53185
entrega : Mayo 2024

Muchas gracias!